Qualcomm Technologies, Inc.

Pro Audio and Low Latency (LL) Audio Playback

80-PB802-2 D

October 31, 2017

Qualcomm Confidential - May Contain Trade Secrets
2022-11-05 10:34:33 GMT
nijiasheng@longcheer.com

Confidential and Proprietary – Qualcomm Technologies, Inc.

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to: DocCtrlAgent@qualcomm.com.

Restricted Distribution: Not to be distributed to anyone who is not an employee of either Qualcomm Technologies, Inc. or its affiliated companies without the express approval of Qualcomm Configuration Management.

Not to be used, copied, reproduced, or modified in whole or in part, nor its contents revealed in any manner to others without the express written permission of Qualcomm Technologies, Inc.

QXDM Professional is a product of Qualcomm Technologies, Inc. Other Qualcomm products referenced herein are products of Qualcomm Technologies, Inc. or its subsidiaries.

Qualcomm is a trademark of Qualcomm Incorporated, registered in the United States and other countries. QXDM Professional is a trademark of Qualcomm Incorporated. Other product and brand names may be trademarks or registered trademarks of their respective owners.

This technical data may be subject to U.S. and international export, re-export, or transfer (“export”) laws. Diversion contrary to U.S. and international law is strictly prohibited.

Qualcomm Technologies, Inc.
5775 Morehouse Drive
San Diego, CA 92121
U.S.A.

© 2017 Qualcomm Technologies, Inc. All rights reserved.

Revision history

Revision	Date	Description
A	March 2017	Initial release
B	June 2017	Numerous changes were made to this document, it should be read in its entirety.
C	June 2017	Updated Section 2.5
D	October 2017	Added Sections 4.1 and 4.5. Updated Sections 2.1, 3.1, 4.2.1, and 4.3.2.

Qualcomm
Confidential - May Contain Trade Secrets
2022-11-05 10:34:33 GMT
nijiasheng@longcheer.com

Contents

1 Introduction 6

1.1 Purpose 6
1.2 Conventions 6
1.3 Technical assistance 6

2 Pro Audio 7

2.1 AudioTrack configuration parameters 8
2.2 Audio HAL interface 8
2.3 Kernel design 9
2.4 TinyMix commands 11
2.5 Customization 11

3 LL playback 12

3.1 AudioTrack configuration parameters 12
3.2 Audio HAL interface 13
3.3 Kernel design 14
3.4 TinyMix commands 16

4 Debugging 17

4.1 Testing Pro Audio 17
4.2 Systrace 17
4.2.1 Collect live traces 17
4.2.2 Collect background traces 18
4.2.3 Add additional traces 19
4.2.4 Analyze traces 19
4.3 Dumpsys 21
4.4 Tee Sink 23
4.5 Debugging ProAudio issues with Wi-Fi on 24
4.6 Enable logs 25
4.6.1 Logcat logs 25
4.6.2 Kernel logs 25
4.6.3 Wi-Fi logging 26

5 Common issues 27

5.1 No audio/mute in Pro Audio playback path in ULL mode 27
5.2 No audio/mute in Pro Audio playback path in ULLPP mode 27
5.3 Audio glitches in Pro Audio/LL playback path 27
5.4 No audio/mute in LL playback 28

Qualcomm Confidential May Contain Trade Secrets 2022-11-05 10:34:33 GMT nijiashang@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6 Log analysis 29
6.1 Log analysis – ULL playback 29
6.1.1 Logcat logs – Setup 29
6.1.2 Logcat logs – Tear down 31
6.1.3 Kernel logs – Setup 31
6.1.4 Kernel logs – Tear down 32
6.2 Log analysis – LL playback 32
6.2.1 Logcat logs – Setup 32
6.2.2 Logcat logs – Tear down 34
6.2.3 Kernel logs – Setup 34
6.2.4 Kernel logs – Tear down 36

A References 37
A.1 Related documents 37
A.2 Acronyms and terms 37

Qualcomm
Confidential - May Contain Trade Secrets
2022-11-05 10:34:33 GMT
nijiasheng@longcheer.com

Figures

Figure 2-1 Pro Audio software architecture when NoIRQ platform driver is used 7
Figure 2-2 Pro Audio playback call flow diagram 10
Figure 3-1 LL playback software architecture when PCM platform driver is used 12
Figure 3-2 LL playback call flow diagram 15

Qualcomm
Confidential - May Contain Trade Secrets
2022-11-05 10:34:33 GMT
nijiasheng@longcheer.com

1 Introduction

1.1 Purpose

This document describes Pro Audio and low-latency (LL) audio playback modes.

1.2 Conventions

Function declarations, function names, type declarations, attributes, and code samples appear in a different font, for example, `#include`.

Shading indicates content that has been added or changed in this revision of the document.

1.3 Technical assistance

For assistance or clarification on information in this document, submit a case to Qualcomm Technologies, Inc. (QTI) at https://createpoint.qti.qualcomm.com/.

If you do not have access to the CDMATech Support website, register for access or send email to support.cdmatech@qti.qualcomm.com.

Qualcomm Confidential - May Contain Trade Secrets
2022-11-05 10:32:33 GMT
nijiasheng@longchear.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2 Pro Audio

The Pro Audio (https://developer.android.com/ndk/guides/audio/audio-latency.html) feature allows audio playback in nearly real time at a very low latency. It is designed to meet the needs of applications that have very strict low latency requirements.

Pro Audio playback uses short buffers and has two modes of operation.

- Ultra-low latency (ULL) mode – No processing occurs in the DSP, so the audio stream manager (ASM) and audio device manager (ADM) are essentially bypassed
- Ultra-low latency performance (ULLPP) mode – Minimal processing occurs in the DSP

If OpenSLES APIs are used, the Pro Audio path is selected if playback satisfies 48 kHz stereo with 16 bit/float sampling rates. Alternatively, native audio track APIs can be used with AUDIO_OUTPUT_FLAG_RAW for 48 kHz stereo with 16 bit/32 bit sampling rates.

Pro Audio prevents GLink interrupts and implements fewer threads, resulting in fewer jitter issues. Pro Audio is typically used in gaming applications because of the strict latency requirements.

Figure 2-1 shows the Pro Audio playback software architecture when the NoIRQ platform driver is used.

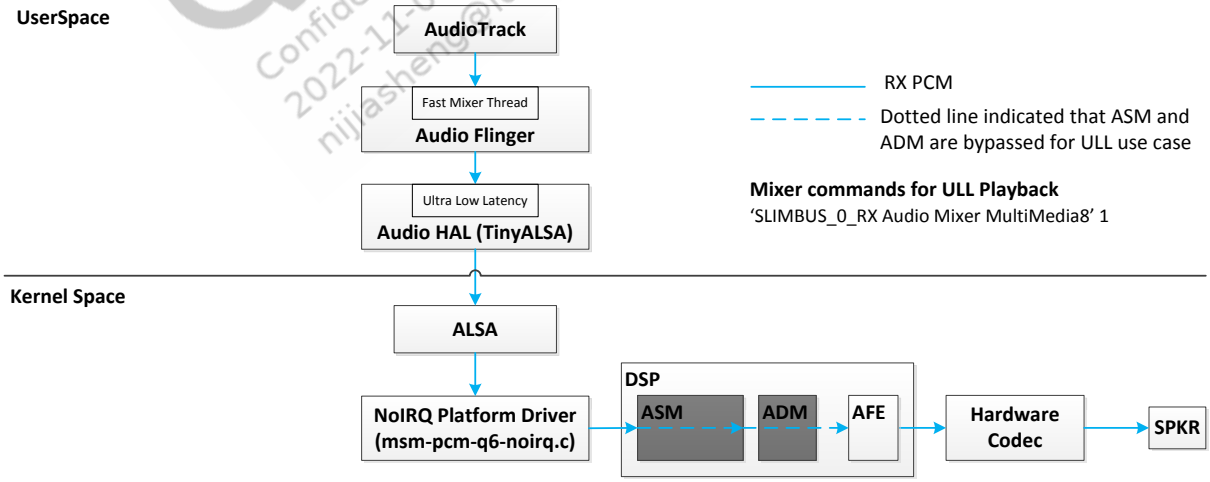


Figure 2-1 Pro Audio software architecture when NoIRQ platform driver is used

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2.1 AudioTrack configuration parameters

When playback starts, an AudioTrack is created with certain parameters. The Pro Audio path is granted/guaranteed by the Android framework only if the following conditions are met:

- Flag requested by the client is AUDIO_OUTPUT_FLAG_RAW and AUDIO_OUTPUT_FLAG_FAST
- Sampling rate – 48 kHz
- Format – 16 bit/float/32 bit
- Channels – Stereo

The ULL use case operates in the fast_mixer thread context of AudioFlinger as there is minimal processing (volume control, mono -> stereo channel configuration) happening in AudioFlinger and, starting with the Android O release, a dedicated writer thread spawned by the HAL service which synchronizes with fast_mixer. Data is exchanged between the two thread contexts via shared memory. The 32 bit format can be used if native audio tracks are used.

If the AudioTrack created does not satisfy the restrictions listed above, playback will fall back to the low latency path or the primary output. The flags requested by client are AUDIO_OUTPUT_FLAG_FAST and AUDIO_OUTPUT_FLAG_RAW. If 32 bit or float format is used for AudioTrack, AudioFlinger will convert the data to 16 bit before rendering to DSP. No effects processing is available in this path.

2.2 Audio HAL interface

Once AudioFlinger opens the output stream, USECASE_AUDIO_PLAYBACK_ULL is selected in the HAL based on the parameters set by the AudioTrack. Next, the buffer sizes for data exchange between the apps processor and the DSP are determined. For ULL, the PCM configuration parameters are defined in hardware/qcom/audio/hal/audio_hw.c:

```
static int af_period_multiplier = 4;
struct pcm_config pcm_config_rt = {
    .channels = 2,
    .rate = DEFAULT_OUTPUT_SAMPLING_RATE,
    .period_size = ULL_PERIOD_SIZE, //1 ms
    .period_count = 512, //=> buffer size is 512ms
    .format = PCM_FORMAT_S16_LE,
    .start_threshold = ULL_PERIOD_SIZE*8,
    .stop_threshold = INT_MAX,
    .silence_threshold = 0,
    .silence_size = 0,
    .avail_min = ULL_PERIOD_SIZE, //1 ms
};
```

- Period size (ULL_PERIOD_SIZE) is hard coded to 48 frames or 1 ms (48000 samples are transferred in 1 sec)
  - Period size, when multiplied by the af_period_multiplier, determines the rate at which the fast_mixer thread or AudioFlinger writes data

Qualcomm Confidential – May Contain Trade Secrets 2022-10-18 10:24:33 GMT niranshen@qualcomm.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

- Buffer size is calculated as period size * af_period_multiplier = 1ms * 4 = 4ms
- Bytes transferred in 1 ms = period size * number of channels * bytes per sample = 48 * 2 (stereo) * 2 (16 bit) = 192 bytes
- Bytes transferred in 4ms = 192 * 4 = 768 bytes

NOTE: The af_period_multiplier is currently capped at 4 ms for Pro Audio playback

After buffers are set, the devices are selected for the ULL use case and a NoIRQ PCM device is opened (pcm_open(). The apps processor is now ready to write the data (pcm_mmap_write()).

The following is the ULL buffer transfer scheme:

- The apps processor continuously writes to a buffer, updating an apps pointer. The DSP continuously reads from the same buffer, updating a hardware pointer.
- After the apps processor finishes the first write (pcm_mmap_write()), it updates the apps pointer and goes to sleeps. The DSP then starts reading. The logic of when the apps processor needs to wake up in order to write the next chunk of data is in libadm.so. In this mechanism, the apps processor does not wait for any event from the DSP to write the next chunk of data to the buffer. Instead, it uses a mechanism to determine where the hardware pointer is at certain instant or checks how much data has been played and wakes up accordingly.

2.3 Kernel design

The mixer_paths.xml file in the user space defines the audio route for the Pro Audio ULL playback mode:

```
<path name="audio-ull-playback">
<ctl name="SLIMBUS_0_RX Audio Mixer MultiMedia8" value="1" />
</path>
```

In the machine driver (defined in kernel/msm-4.4/sound/soc/msm/msm8998.c), the front end (FE) Multimedia8 is associated with the NoIRQ platform driver (sound/soc/msm/qdsp6v2/msm-pcm-q6-noirq.c). The NoIRQ platform driver is selected in the kernel because there are no interrupt requests between the apps processor and the DSP in ULL/ULLPP mode.

The msm_pcm_pointer() API in the NoIRQ driver queries the position of the DSP through q6asm_get_shared_pos() API. The q6asm_get_shared_pos() API gets the read index of the DSP in bytes, converts it to frames, and updates the hardware pointer. This information is used by the apps pointer to determine when the app processor should wake up.

Qualcomm Confidential – May Contain Trade Secrets 2022-11-05 10:34:33 GMT nijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Figure 2-2 shows the Pro Audio playback call flow.

Figure 2-2 Pro Audio playback call flow diagram

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:33 GMT nijasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2.4 TinyMix commands

To perform Pro Audio playback over the SLIMbus interface:

```
tinymix “SLIMBUS_0_RX Audio Mixer MultiMedia8" 1
tinyplay /data/test.wav
```

To perform ULL playback over the primary MI2S interface:

```
tinymix “PRI_MI2S_RX Audio Mixer MultiMedia8" 1
tinyplay /data/test.wav
```

2.5 Customization

The following customizations can be configured for Pro Audio playback:

- To add minimal processing using ULLPP mode (still guarantees strict low latency requirements for Pro Audio), select the topology ID associated with ULLPP mode before the pcm_open() API is called for the NoIRQ device. The different topologies available can be referenced in sound/soc/msm/qdsp6v2/msm-pcm-q6-noirq.c. Refer to the following CAF link for the mixer controls used to add the ULLPP use case support for Pro Audio:
  - https://source.codeaurora.org/quic/la/platform/hardware/qcom/audio/commit/?id=5e4ccbc19eaf2496bbe87e5a73996a5cf69d1805

  To dynamically select ULLPP:
  - https://source.codeaurora.org/quic/la/kernel/msm-4.4/commit/?id=c16632d1d6c17e78358f70800a1e832d96641603
  - https://source.codeaurora.org/quic/la/kernel/msm-4.4/commit/?h=LE.UM.1.1.4.r1&id=025745f82d0a505ef17d53decc50cb10e21bb707

Qualcomm Confidential and Proprietary – Qualcomm Trade Secrets 2014-11-05 04:34:51 UTC
jijia@qcom.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3 LL playback

LL audio playback is played with minimum delay. This is achieved by using short buffers and minimal processing in the DSP. LL playback is typically used for touch tones, alert tones, ringtones, and notifications. Algorithms which have very high latency in the DSP cannot be used in this mode.

Figure 3-1 shows the LL playback software architecture when the PCM platform driver is used.

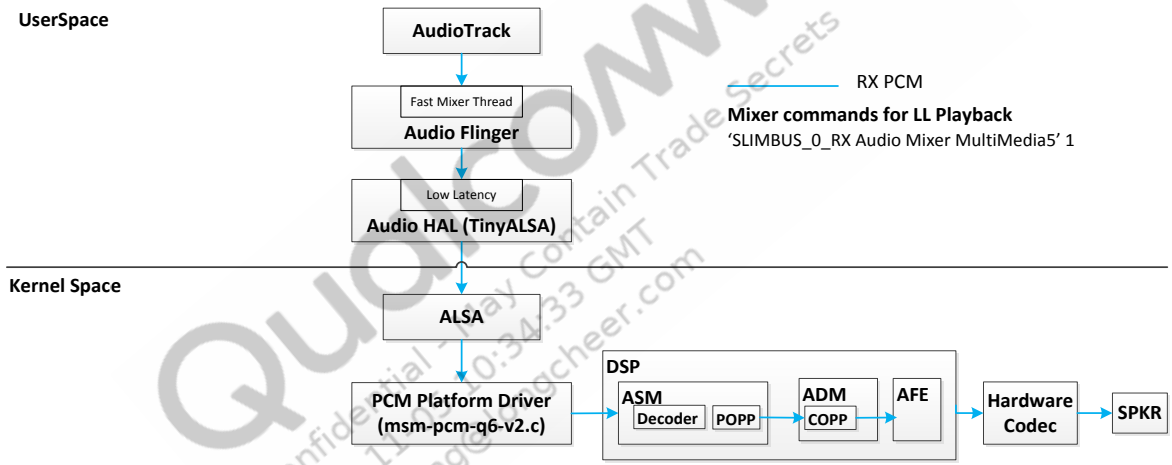


Figure 3-1 LL playback software architecture when PCM platform driver is used

3.1 AudioTrack configuration parameters

The LL path is granted by the Android framework only if the following conditions are met:

- Flag requested by the client is AUDIO_OUTPUT_FLAG_FAST or AUDIO_OUTPUT_FLAG_PRIMARY
- Sampling rate – 48 kHz
- Format – 16 bit/float
- Channels – Stereo

The LL use case operates in the fast_mixer thread context of AudioFlinger as there is minimal processing happening in AudioFlinger and, starting with the Android O release, a dedicated writer thread spawned by the HAL service which synchronizes with fast_mixer. Data is exchanged between the two thread contexts via shared memory.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3.2 Audio HAL interface

Once AudioFlinger opens the output stream, USECASE_AUDIO_PLAYBACK_LOW_LATENCY is selected in the HAL based on the parameters set by the AudioTrack. Next, the buffer sizes for data exchange between the apps processor and the DSP are determined. For LL, the PCM configuration parameters are defined in hardware/qcom/audio/hal/audio_hw.c:

```
struct pcm_config pcm_config_low_latency = {
    .channels = 2,
    .rate = DEFAULT_OUTPUT_SAMPLING_RATE,
    .period_size = LOW_LATENCY_OUTPUT_PERIOD_SIZE,
    .period_count = LOW_LATENCY_OUTPUT_PERIOD_COUNT,
    .format = PCM_FORMAT_S16_LE,
    .start_threshold = LOW_LATENCY_OUTPUT_PERIOD_SIZE / 4,
    .stop_threshold = INT_MAX,
    .avail_min = LOW_LATENCY_OUTPUT_PERIOD_SIZE / 4,
};
```

- Period size for LL is hard coded to:

  ```
  PRODUCT_PROPERTY_OVERRIDES += \
  audio_hal.period_size=192
  ```

- Period size = 192 frames or 4 ms (48 frames for 1 ms)
- Buffer size = 4 ms
- Bytes transferred in 4 ms = period size * number of channels * bytes per sample = 192 * 2 (stereo) * 2 (16 bit) = 768 bytes
- Bytes transferred in 4 ms = 192 * 4 = 768 bytes
- The period count is 2, indicating that the ring buffer has a size of 1536 bytes (768 * 2)

After buffers are set, the devices are selected for the LL use case and a PCM device is opened (pcm_open()). The apps processor is now ready to write the data (pcm_write()).

The following is the LL buffer transfer scheme:

- The apps processor continuously writes to the ring buffer and the DSP reads the buffer
- The DSP sends an event back to the apps processor indicating that is has consumed the buffer
- The DSP frees the buffer it has read so the apps processor has a free buffer in which to write the next chunk of data. This triggers the next pcm_write.

Qualcomm Confidential - May Contain Trade Secrets 2020-11-05 20:34:33 GMT nijia.hong@lomscheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3.3 Kernel design

The mixer_paths.xml file in the user space defines the audio route for the LL use case:

```
<path name="low-latency-playback">
<ctl name="SLIMBUS_0_RX Audio Mixer MultiMedia5" value="1" />
</path>
```

In the machine driver (defined in kernel/msm-4.4/sound/soc/msm/msm8998.c), the FE Multimedia5 is associated with the PCM platform driver (sound/soc/msm/qdsp6v2/msm-pcm-q6-v2.c) and the PCM platform driver is selected in the kernel. The mechanism implemented in the PCM driver is:

- The apps processor fills 1536 bytes worth of data (8 ms) and notifies the DSP using a write event. This event is sent across to the DSP over GLINK.
- The DSP begins to read the data and, once it consumes the first buffer, sends the write_done event back to the apps processor indicating that is has consumed the buffer. The write_done event is transferred over GLINK. The buffer is freed up and the DSP positon is updated to the start of the second buffer at the 768 bytes position.
- The apps processor writes to the buffer just freed up by the DSP. Once the DSP reads the buffer, it once again sends an event to the apps processor indicating that it has consumed the buffer and frees up the buffer by updating the DSP position to the start of the first buffer at the 0 bytes position. This allows the apps processor to write into the second buffer, which has just been freed up.
- The process repeats until playback ends.
- The kernel communicates with the DSP via APR over GLINK. For more details on GLINK, refer to the following documents:
  - *Linux SMD GLink Migration Guide* (80-VJ608-2)
  - *Non-HLOS SMDL To GLink Migration Guide* (80-VJ608-3)
  - *GLink (Generic Link) Overview* (80-P2598-1)
  - Details on the Linux side available at /kernel/msm-4.4/Documentation/arm/msm/glink.txt

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:37 GMT xiaosheng@tongcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Figure 3-2 shows the LL playback call flow.

Figure 3-2 LL playback call flow diagram

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:33 GMT nijiashang@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3.4 TinyMix commands

To perform LL playback over the SLIMbus interface:

```
tinymix “SLIMBUS_0_RX Audio Mixer MultiMedia5" 1
tinyplay /data/test.wav
```

To perform LL playback over the primary MI2S interface:

```
tinymix “PRI_MI2S_RX Audio Mixer MultiMedia5" 1
tinyplay /data/test.wav
```

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:33 GMT nijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4 Debugging

4.1 Testing Pro Audio

NOTE: This section was added to this document revision.

To test Pro Audio:

1. Push a file (for example, test.wav) to the sdcard location of your device.
2. Open the chrome browser on your device and play the file:

   file:///sdcard/Music/test.wav

4.2 Systrace

The following prerequisites apply:

- For Windows 7
  - Platform tools – https://developer.android.com/studio/releases/platform-tools.html
  - Python 2.7
- For Linux
  - Use Systrace from the build path – external/chromium/trace/systrace.py
  - Python 2.7

General information about Systrace is available at https://developer.android.com /studio/profile/systrace-commandline.html.

4.2.1 Collect live traces

Use the following command to collect Systraces live:

```
>> <PATH>\python.exe <PATH>\systrace.py" tag1 tag2 … tagN -t XX -o
test.html
```

The arguments in this command represent the following:

- tag1 tag2 …. tagN – Represent the components in which to enable Systraces
- -t – Represents the duration for which to collect the Systrace
- -o – Represents the output Systrace html file

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:23 GMT xijiasheng@longcheer.com

Example

```
C:\Python27\python.exe "C:\Program Files (x86)\Android\android-sdk\platform-tools\systrace\systrace.py" sched freq audio irq -t 30 –o trace_output.html
```

In this example:

- sched – Enables sched traces, which provide information on the thread being scheduled in/scheduled out and on which core it is being scheduled
- freq – Enables freq traces, which provide the frequency of the CPUs
- audio – Enables audio traces
- irq – Enables irq traces
- 30 – Duration of the trace (in this case, 30 seconds)
- trace_output.html – Name of the output Systrace file (created when the command is run)

NOTE: The -b argument may need to be used in cases where the buffer size needs to be increased. If a collected trace contains missing trace points, the buffer size could be potentially increased using the -b argument. Typically, if the scheduler traces are run for a long duration, the Systrace buffer inside the memory will be overwritten, causing traces to be misses.

4.2.2 Collect background traces

Background tracing is useful for issues where the user does not know when the issue will occur (for example, if a user is testing and hears a glitch with dialer tones). To collect background traces:

1. Before starting the test, run the following command to start the trace asynchronously:

   ```
   adb shell "atrace --async_start -z -c -b 8000 sched audio"
   ```

   In this command:

   - -z – Compresses the trace
   - -c – Circular buffer that will be overwritten
   - -b – Buffer size that will be captured
   - The trace is being collected for scheduler and audio (in this case)

2. Reproduce the problem. When the event of interest occurs, run the following command to dump the trace into the sdcard/touchtone.trace file:

   ```
   adb shell "atrace --async_dump -z -c -b 80000 sched audio > /sdcard/touchtone.trace" download trace for the last 10 seconds
   ```

3. Run the following command to pull the trace file:

Qualcomm Confidential and Proprietary May contain Trade Secrets 2022-11-03 10:34:33 GMT nijiacheng@dongcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
adb pull /sdcard/touchtone.trace
```

4. Run the following command to stop trace collection:

```
adb shell "atrace --async_stop“
```

5. Open the trace file using the Chrome browser.

If the issue is not seen in the traces collected, tweak the buffer size based on the use case.

4.2.3 Add additional traces

NOTE: Numerous changes were made in this section.

Additional event traces can be enabled as follows:

```
echo irq:* >> /sys/kernel/debug/tracing/set_event
echo ipi:* >> /sys/kernel/debug/tracing/set_event
echo timer:* >> /sys/kernel/debug/tracing/set_event
echo sched:* >> /sys/kernel/debug/tracing/set_event
```

To add additional traces in the HAL, contact the customer engineering team as described in Section 1.3.

4.2.4 Analyze traces

Use the following keyboard shortcuts when analyzing traces:

- W – Zoom in
- S – Zoom out
- A – Move left
- D – Move right

For audio issues, AudioOut_D and the FastMixer thread should be analyzed in Systrace. To perform Systrace analysis:

1. In the FastMixer thread, check if all writes are happening at 4 ms or not in FastMixer thread. Based on empirical data, if writes take more than 5 ms, there could be a problem observed as an audio glitch.
2. Ensure that no gaps exist between consecutive writes that are longer than 1-2 ms. Such gaps can correspond to jitter and be observed as an audio glitch.
3. Identify the issue using the timestamps. In the example below, at 1494.018 ms there is a gap of 3.225 ms and the task is runnable.
4. Open the Systrace in a text editor to identify what happened at the identified timestamp.

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:33 GMT mjiasheng@longsheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

5. Search for “entries-in-buffer” to find the start of the trace time. In the example below, the start time is 1405.270162. Ftrace specifies the kernel time in seconds (in this case, Systrace starts at 0).

```
# tracer: nop
#
#entries-in-buffer/entries-written: 125593/125593   #P:8
#
#                                       _-----=> irqs-off
#                                      / _----=> need-resched
#                                     | / _---=> hardirq/softirq
#                                     || / _--=> preempt-depth
#                                     ||| /     delay
#           TASK-PID    TGID   CPU#  ||||    TIMESTAMP  FUNCTION
#              | |        |      |   ||||       |         |
irq/693-dwc3-410 ( 410) [003] d..3  1405.270162: sched_switch:
prev_comm=irq/693-dwc3 prev_pid=410 prev_prio=49 prev_state=S ==>
next_comm=->transport next_pid=5301 next_prio=120
```

6. In this example, a gap has been identified at 1.494018s. Add this to the ftrace start time (1405.270162s + 1.494018s = 1406.76418s) to find out what happened at that time stamp**.**
7. At 1406.76418s in ftrace, AudioOut is scheduled out.

```
AudioOut_D-1557  ( 1008) [001] ...1  1406.764072: tracing_mark_write: E
kworker/0:1-5089  ( 5089) [000] d..5  1406.764104: sched_wakeup:
comm=SDM_EventThread pid=918 prio=111 target_cpu=001
kworker/0:1-5089  ( 5089) [000] d..3  1406.764132: sched_switch:
prev_comm=kworker/0:1 prev_pid=5089 prev_prio=120 prev_state=S ==>
next_comm=kworker/u16:22 next_pid=5161 next_prio=120
          <idle>-0     (-----) [002] dnh4  1406.764152: sched_wakeup:
comm=DispSync pid=829 prio=97 target_cpu=001
AudioOut_D-1557  ( 1008) [001] d..3  1406.764180: sched_switch:
prev_comm=AudioOut_D prev_pid=1557 prev_prio=101 prev_state=R ==>
next_comm=DispSync next_pid=829 next_prio=97
          <idle>-0     (-----) [002] d..3  1406.764186: sched_switch:
prev_comm=swapper/2 prev_pid=0 prev_prio=120 prev_state=R ==>
next_comm=mdss_fb0 next_pid=5210 next_prio=83
android.ui-1793  ( 1737) [003] d..5  1406.764206: sched_wakeup:
comm=InputDispatcher pid=2100 prio=112 target_cpu=001
kworker/u16:22-5161 ( 5161) [000] d..4  1406.764215: sched_wakeup:
comm=kworker/u16:7 pid=5141 prio=120 target_cpu=001
```

After approximately 3 ms, AudioOut comes back in.

```
Binder:1008_2-2126  ( 1008) [000] d..3  1406.767367: sched_switch:
prev_comm=Binder:1008_2 prev_pid=2126 prev_prio=110 prev_state=S ==>
next_comm=.android.dialer next_pid=4927 next_prio=110
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
com.android.dialer-4927  ( 4927) [000] d..3  1406.767395: sched_switch:
prev_comm=.android.dialer prev_pid=4927 prev_prio=110 prev_state=S ==>
next_comm=swapper/0 next_pid=0 next_prio=120
/system/bin/surfaceflinger-777   (  777) [001] d..3  1406.767405:
sched_switch: prev_comm=surfaceflinger prev_pid=777 prev_prio=98
prev_state=S ==> next_comm=AudioOut_D next_pid=1557 next_prio=101
AudioOut_D-1557  ( 1008) [001] d..3  1406.767458: sched_wakeup:
comm=Binder:1008_2 pid=2126 prio=110 target_cpu=000
          <idle>-0     (-----) [003] d..3  1406.767468: sched_switch:
prev_comm=swapper/3 prev_pid=0 prev_prio=120 prev_state=R ==>
next_comm=RenderThread next_pid=4952 next_prio=110
          <idle>-0     (-----) [000] d..3  1406.767474: sched_switch:
prev_comm=swapper/0 prev_pid=0 prev_prio=120 prev_state=R ==>
next_comm=Binder:1008_2 next_pid=2126 next_prio=110
mdss_fb0-5210  ( 5210) [002] d..3  1406.767519: sched_switch:
prev_comm=mdss_fb0 prev_pid=5210 prev_prio=83 prev_state=D ==>
next_comm=swapper/2 next_pid=0 next_prio=120
AudioOut_D-1557  ( 1008) [001] ...1  1406.767532: tracing_mark_write:
B|1008|write
Binder:1008_2-2126  ( 1008) [000] dn.6  1406.767552: sched_wakeup:
comm=.android.dialer pid=4927 prio=110 target_cpu=000
```

8. In this example, the correct action at this point is to contact the scheduler team to check why the AudioThread was scheduled out when it was in runnable state and other threads were scheduled in. Provide the scheduler team with Systraces noting the exact timestamps of the identified gap and any further analysis.

If the issue is not easily evident in the Systrace logs:

- The issue may still be an audio issue – Contact the audio development team. Provide all Systraces, the timestamps of identified gap, and any further analysis.
- At the timestamp of the glitch, check if the CPU is scaling. If the CPU is scaling to handle the load of the system, it may cause a glitch as the client may not provide the buffer in time. In this case, FastMixer output will not have any issues and gaps in the writes indicates they will be underrun in the client. In ftrace, the CPU number is displayed in square brackets – Check if the CPU number changes for AudioOut. This may indicate the glitch is a result of the amount of time it takes to move between audio cores.

4.3 Dumpsys

After an issue occurs, collect Dumpsys logs using the following command:

```
adb shell dumpsys media.audio_flinger
```

Dumpsys:

- Can be redirected to a particular file
- Is useful for knowing that an issue occurred, but does not provide information on why the issue happened

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

- Can be used to check for underruns. If underruns exist, glitches may be observed. If the std_dev increases, a glitch will be observed.

Example – Pro Audio output

```
AudioStreamOut: 0xf2f49618 flags 0x104 (FAST|RAW)
  Frames written: 0
  Suspended frames: 0
  PipeSink frames written: 0
  Hal stream dump:
  Thread throttle time (msecs): 0
  AudioMixer tracks: 0x00000000
  Master mono: off
  FastMixer command=COLD_IDLE writeSequence=0 framesWritten=0
            numTracks=0 writeErrors=0 underruns=0 overruns=0
            sampleRate=0 frameCount=0 measuredWarmup=0 ms, warmupCycles=0
            mixPeriod=nan ms
  No FastMixer statistics available currently
  Fast tracks: sMaxFastTracks=8 activeMask=0
  Index Active Full Partial Empty  Recent Ready    Written
      0     no    0       0     0    full    0          0
      1     no    0       0     0    full    0          0
      2     no    0       0     0    full    0          0
      3     no    0       0     0    full    0          0
      4     no    0       0     0    full    0          0
      5     no    0       0     0    full    0          0
      6     no    0       0     0    full    0          0
      7     no    0       0     0    full    0          0
```

Example – LL playback output

```
AudioStreamOut: 0xf2f49500 flags 0x6 (PRIMARY|FAST)
  Frames written: 4795200
  Suspended frames: 0
  PipeSink frames written: 4795200
  Hal stream dump:
  Thread throttle time (msecs): 0
  AudioMixer tracks: 0x00000007
  Master mono: off
  FastMixer command=MIX_WRITE writeSequence=50083 framesWritten=4807872
            numTracks=1 writeErrors=0 underruns=1 overruns=0
            sampleRate=48000 frameCount=192 measuredWarmup=18.4 ms,
warmupCycles=5
            mixPeriod=4.00 ms
  Simple moving statistics over last 3.0 seconds:
    wall clock time in ms per mix cycle:
```

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:28 GMT nijiashengqiongzhier.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
    mean=4.00 min=3.25 max=4.74 stddev=0.21
  raw CPU load in us per mix cycle:
    mean=457 min=0 max=1078 stddev=195
Fast tracks: sMaxFastTracks=8 activeMask=0x1
Index Active Full Partial Empty  Recent Ready    Written
    0    yes  396       0     0    full  1728    4794432
    1     no  759      33   383   empty     0     148896
    2     no  387      19   510   empty     0      98816
    3     no    0       0     0    full     0          0
    4     no    0       0     0    full     0          0
    5     no    0       0     0    full     0          0
    6     no    0       0     0    full     0          0
    7     no    0       0     0    full     0          0
```

4.4 Tee Sink

AudioFlinger keeps an in-memory copy of buffers written from FastMixer. For cases where a glitch may be caused before Audio Flinger, creating a Tee Sink can be useful for determining the cause of the issue.

After the Tee Sink, collect Dumpsys logs along with the statistics. A .wav file corresponding to FastMixer is produced – Use this to determine if the glitch can be heard/introduced before writing to the HAL. If the glitch is a result of a timing issue, the dump will show no errors but a glitch is still heard.

To create a tee sink:

1. In `frameworks/av/services/audioflinger/Configuration.h`, uncomment `#define TEE_SINK` to enable the tee sink.
2. After enabling, recompile AudioFlinger.
3. Push audioflinger libraries (both 64 and 32 bit variants) to /system/lib64 and /system/lib, respectively.
4. Run the following commands:

```
adb shell setprop ro.debuggable 1
adb shell setprop af.tee 7
adb shell stop audioserver
adb shell start audioserver
```

Setting af.tee helps to debug input, output, and data given to the AudioTrack. Check http://opengrok.qualcomm.com/source/xref/LA.UM.6.4/frameworks/av/services/audioflinger/AudioFlinger.cpp#192 for more details on the flags.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

5. If af.tee includes a debugging track (any bitmask that includes 4), then the last 10 secs of data given to the AudioTrack will be dumped when the track is deleted.

   To dump all data given by the client in AudioTrack during run time (instead of only the default 10 secs), move the code for dumping from http://opengrok.qualcomm.com /source/xref/LA.UM.6.4.9/frameworks/av/services/audioflinger/Tracks.cpp#236 (from ~TrackBase) to http://opengrok.qualcomm.com/source/xref/LA.UM.6.4.9/ frameworks/av/services/audioflinger/Tracks.cpp#498 (inside ::dump) in Tracks.cpp.

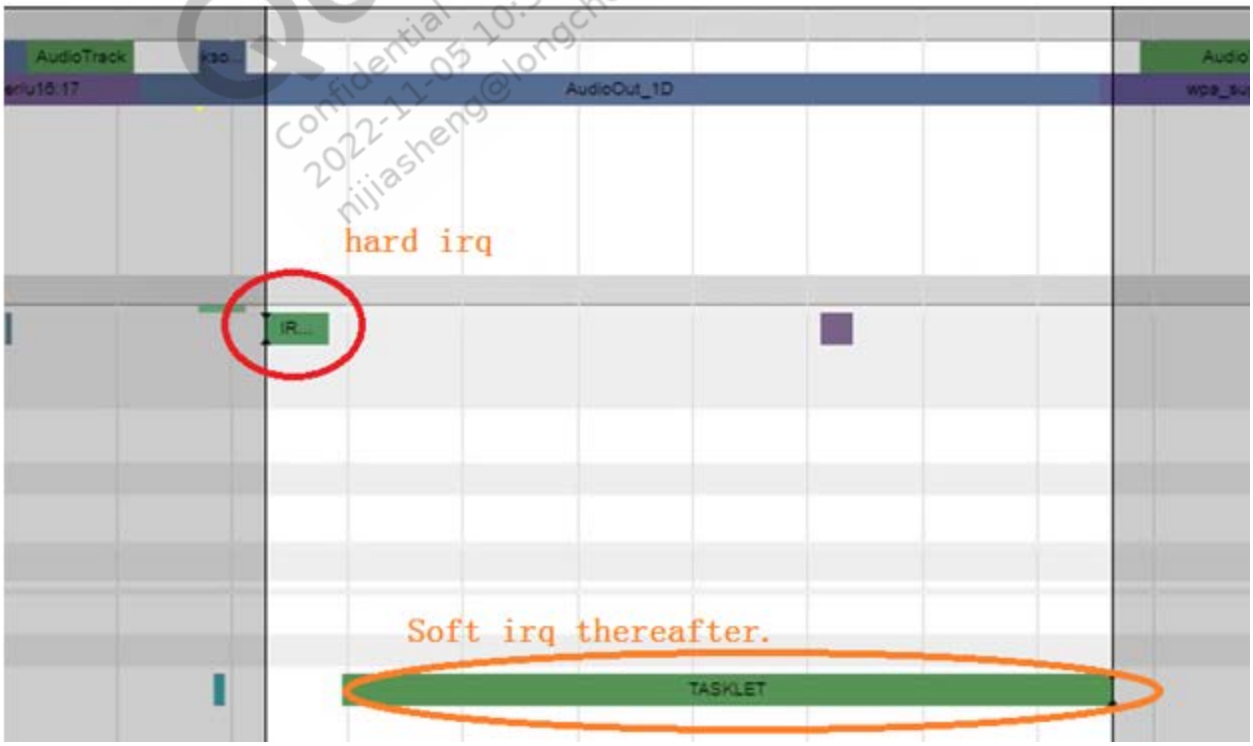
4.5 Debugging ProAudio issues with Wi-Fi on

NOTE: This section was added to this document revision.

Determine if the QTI Wi-Fi driver or a 3rd party Wi-Fi driver is being used. Confirm with OEMs respective WLAN team to check which WLAN/ Wi-Fi driver is being used.

- If the QTI Wi-Fi driver is being used, collect traces as described in Section 4.2 and contact the customer engineering team at https://support.cdmatech.com.
- If a 3rd party Wi-Fi driver is being used, identify the timestamp of the glitch and analyze potential problems.

  For example, in the example shown below, the glitch occurs near the 4831.3 ms timestamp. Hardware IRQ is observed in core 0 (same core as the fast_mixer thread), followed by a soft IRQ/taslet running for a long time (>1 ms), which could contribute to the glitch. In such a case, the 3rd party Wi-Fi driver provider needs to optimize from their end.



Qualcomm Confidential - May contain Trade Secrets 2022-11-05 10:34:33 GMT nijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.6 Enable logs

4.6.1 Logcat logs

Run the following command to enable Logcat logs:

```
adb logcat –c  && adb logcat –v threadtime  | tee logcat.txt
```

Depending on the issue, the following additional logcat logs that can be enabled by uncommenting `#define LOG_NDEBUG 0`:

- /frameworks/av/media/libmedia/AudioTrack.cpp
- /hardware/qcom/audio/hal/audio_hw.c
- /hardware/qcom/audio/hall/msm8974/platform.c
- /frameworks/av/services/audioflinger/AudioFlinger.cpp
- /hardware/qcom/audio/policy_hal/AudioPolicyManager.cpp

4.6.2 Kernel logs

For Pro Audio playback, run the following command to enable kernel logs:

```
adb shell "echo file msm-pcm-routing-v2.c +p > /sys/kernel/debug/
dynamic_debug/control"
adb shell "echo file msm-pcm-q6-noirq.c +p > /sys/kernel/debug/
dynamic_debug/control"
```

For LL playback, run the following command to enable kernel logs:

```
adb shell "echo file msm-pcm-routing-v2.c +p >
/sys/kernel/debug/dynamic_debug/control"
adb shell "echo file msm-pcm-q6-v2.c+p >
/sys/kernel/debug/dynamic_debug/control"
```

Depending on the issue, additional logs can be enabled by running the following commands:

```
adb shell "echo file q6afe.c +p > /sys/kernel/debug/dynamic_debug/control"
adb shell "echo file q6asm.c +p > /sys/kernel/debug/dynamic_debug/control"
adb shell "echo file q6adm.c +p > /sys/kernel/debug/dynamic_debug/control"
adb shell "echo file msm-dai-q6-v2.c +p > /sys/kernel/debug/
dynamic_debug/control"
adb shell "echo file msm8998.c +p > /sys/kernel/debug/
dynamic_debug/control"
```

Qualcomm Confidential May Contain Trade Secrets 2022-11-15 10:24:33 GMT nijasheng@zongchuer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.6.3 Wi-Fi logging

To enable Wi-Fi logging:

1. Ensure that Wi-Fi is enabled on the device and that it is connected to the same network as the host computer.
2. Connect the USB debugging cable to the device.
3. Run the following command from the host computer to display IP configuration information:

```
adb shell ifconfig
wlan0   Link encap:Ethernet HWaddr 58:48:22:a0:5b:78
inet addr:10.73.182.40 Bcast:10.73.183.255 Mask:255.255.254.0
inet6 addr: fe80::5a48:22ff:fea0:5b78/64 Scope: Link
UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
RX packets:34748 errors:0 dropped:9 overruns:0 frame:0
TX packets:7192 errors:0 dropped:0 overruns:0 carrier:0    collisions:0
txqueuelen:1000
RX bytes:6033662 TX bytes:2475656
```

4. After finding the IP address of the device, run the following command on the host computer to restart the adb in TCP mode:

```
adb tcpip 5555
```

5. Run the following command to connect adb using the device IP address and port ID:

```
adb connect <IP Address of the Device >:5555
```

6. Remove the USB debugging cable from the device.

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:33 GMT rijashenv@lomscheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

5 Common issues

When reporting a performance issue, ensure the following:

- Performance issue is reported using secondary images/perf images
- No QXDM logging is enabled when the issue is observed and when Systrace is collected
- No additional logging is enabled by QTI or the OEM in the code
- No data is being dumped in the code

If the steps indicated above are not followed, performance issues will be seen due to priority inversions.

5.1 No audio/mute in Pro Audio playback path in ULL mode

1. Verify if the TinyMix/TinyPlay commands described in Section 2.4 work.
2. If the TinyMix/TinyPlay commands work, the issue could be related to a user space component. Enable further logging to check if the configuration in the user space is as expected for ULL mode.

5.2 No audio/mute in Pro Audio playback path in ULLPP mode

1. Verify that the ACDB calibration saved for the ACDB ID corresponds to the device being used for playback.
2. Verify that the App Type ID in audio_policy_configuration.xml matches the topology ID in the ACDB file.

5.3 Audio glitches in Pro Audio/LL playback path

Glitches may occur in the Pro Audio/LL playback use cases as well as concurrency use cases for the following reasons:

- Other threads are being executed at the same time
- Audio threads are in the runnable state but are not running
- Tasks are migrating from one core to the other
- Tasks are disabling interrupts

A good starting point for most audio glitch issues it to collect Systraces, identify the timestamp of the glitch and contact the audio development and scheduler teams as described in Section 4.1.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

To debug audio glitch issues in the Pro Audio/LL playback path:

1. Enable traces in the ADM library using the following commands

   ```
   adb shell setprop audio.adm.enable.traces true
   adb kill-server
   ```

   This property does not persist across reboots.

2. Collect Systrace and Dumpsys simultaneously with instructions provided in Sections 4.1 and 4.3.
3. Check if any underruns are present in the Dumpsys. If underruns are present, a glitch will be observed. Underruns usually occur if the hardware pointer gets ahead of the apps pointer. This indicates that the AudioTrack client (app, key tone generator, etc.) may not be writing data in time, causing the DSP to read data that is not available.
4. Use the check_ptr_margins function in the ADM library to locate the underrun. If the margin/diff between the apps pointer and the hardware pointer is less than zero, look for a log message resembling the following:

   ```
   01-12 16:01:07.783 W 720 1335 qc_adm : margin check:
   adm_request_focus_v2(p) failed diff -144
   ```

   "failed diff - 144" indicates a scheduling jitter of 3 ms – Which means that if audio.adm.buffering.ms was set to 1 ms, a glitch of 2 ms or less will be observed.

5. Identify possible timestamps/gaps that could potentially be the cause of the glitch.
6. Contact the customer engineering team – See Section 1.3.

5.4 No audio/mute in LL playback

1. Verify that the TinyMix commands described in Section 3.4 work on the interface.
2. If the TinyMix/TinyPlay commands work, the issue could be related to a user space component. Enable further logging to check if the configuration in the user space is as expected for LL playback.

Qualcomm Confidential – May Contain Trade Secrets 2022-11-07 10:32:33 GMT nikashenka@longdear.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6 Log analysis

6.1 Log analysis – ULL playback

The following log snippets indicate what to look for in the logcat and kernel logs.

6.1.1 Logcat logs – Setup

6.1.1.1 Audio track creation

```
03-09 04:09:13.881  5298  6655 V AudioTrack: set(): streamType 3,
sampleRate 48000, format 0x1, channelMask 0x3, frameCount 0, flags #104,
notificationFrames -2, sessionId 129, transferType 0, uid -1, pid -1
//StreamType: 3
(/system/media/audio/include/system/audio.h(audio_stream_type_t)
SampleRate: 48000/48kHz
Format: 0x1 (/system/media/audio/include/system/audio.h (audio_format_t))
Channels: 0x3
(/system/media/audio/include/system/audio.h(audio_channel_mask_t))
Flags: 104
(/system/media/audio/include/system/audio.h(audio_output_flags_t))
03-09 04:09:13.898  2153  2294 V AudioPolicyManagerCustom:
getNewOutputDevice() selected device 2
//Output device gets selected as speaker in APM
(/system/media/audio/include/system/audio.h(audio_device_type_t))
```

6.1.1.2 Audio HAL

```
03-09 04:09:13.901  2153  2481 D audio_hw_primary: start_output_stream:
enter: stream(0xf3ed3000)usecase(12: audio-ull-playback) devices(0x2)
//Output Stream selects use case as “audio_ull_playback” per the code below
in adev_open_output_stream() (defined in
/hardware/qcom/audio/hal/audio_hw.c)
if (out->flags & AUDIO_OUTPUT_FLAG_RAW) {
            out->usecase = USECASE_AUDIO_PLAYBACK_ULL;
            out->realtime = may_use_noirq_mode(adev,
USECASE_AUDIO_PLAYBACK_ULL,out->flags);
            out->config = out->realtime ? pcm_config_rt :
pcm_config_low_latency;
out->usecase gets set to USECASE_AUDIO_PLAYBACK_ULL which corresponds to
“audio_ull_playback” in use_case_table
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
out->realtime returns true in this case.

//Checks whether there is a NoIRQ PCM device available and adm library
access.

if (adev->adm_is_noirq_avail)
        return adev->adm_is_noirq_avail(adev->adm_data,
                                        adev->snd_card, dev_id, dir);

03-09 04:09:13.901  2153  2481 D audio_hw_primary: start_output_stream:
enter: stream(0xf3ed3000)usecase(12: audio-ull-playback) devices(0x2)
03-09 04:09:13.903  2153  2481 D audio_hw_primary: select_devices for use
case (audio-ull-playback)
03-09 04:09:13.903  2153  2481 V msm8974_platform:
platform_get_output_snd_device: enter: output devices(0x2)
03-09 04:09:13.903  2153  2481 V msm8974_platform:
platform_get_output_snd_device: exit: snd_device(speaker)
03-09 04:09:13.903  2153  2481 D audio_hw_primary: select_devices:
out_snd_device(2: speaker) in_snd_device(0: )
03-09 04:09:13.904  2153  2481 D audio_hw_primary: enable_snd_device:
snd_device(2: speaker)
03-09 04:09:13.927  2153  2481 V audio_hw_primary: enable_audio_route:
enter: usecase(12)
03-09 04:09:13.928  2153  2481 D audio_hw_primary: enable_audio_route:
apply mixer and update path: audio-ull-playback
03-09 04:09:13.930  2153  2481 V audio_hw_primary: enable_audio_route: exit
03-09 04:09:13.930  2153  2481 D audio_hw_primary: select_devices: done

// For the use-case audio-ull-playback select_devices the output device
gets selected as 0x2 from platform_get_output_snd_device
snd_device = SND_DEVICE_OUT_SPEAKER
select_devices enables the device (enable_snd_device)
select_devices also enables the audio route for audio_ull_playback

03-09 04:09:13.930  2153  2481 V audio_hw_primary: start_output_stream:
Opening PCM device card_id(0) device_id(17) format(0)
//Opening PCM Device 17 (Takes PCM device value from
hardware/qcom/audio/configs/msm8998/audio_platform_info.xml)

03-09 04:09:13.934  2153  2481 V audio_hw_primary: start_output_stream:
pcm_prepare
03-09 04:09:13.994  2153  2481 D audio_hw_primary: start_output_stream:
exit
//Started the output stream successfully and starts buffer exchange

03-09 04:09:13.994  2153  2481 V audio_hw_primary: out_write: writing
buffer (0 bytes) to pcm device
03-09 04:09:13.996  2153  2480 V audio_hw_primary: out_write: writing
buffer (768 bytes) to pcm device
```

Qualcomm Confidential – May Contain Trade Secrets 2022-11-15 10:34:33 GMT jhestrenge@qualcomm.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
03-09 04:09:14.002  2153  2480 V audio_hw_primary: out_write: writing
buffer (768 bytes) to pcm device
03-09 04:09:14.006  2153  2480 V audio_hw_primary: out_write: writing
buffer (768 bytes) to pcm device
03-09 04:09:14.010  2153  2480 V audio_hw_primary: out_write: writing
buffer (768 bytes) to pcm device
// Buffers are transferred every 4 ms
```

After every write, the apps processor determines how long to sleep based on the mechanism described in Section 2.2. In each instance, the actual sleep time may vary depending on how much time it takes the apps processor to wake up (based on the scheduler and other tasks running on the system at that instant).

6.1.2 Logcat logs – Tear down

```
03-11 01:14:01.170  2226  2748 V audio_hw_primary: stop_output_stream:
enter: usecase(12: audio-ull-playback)
03-11 01:14:01.170  2226  2748 V audio_hw_primary: disable_audio_route:
enter: usecase(12)
03-11 01:14:01.170  2226  2748 D audio_hw_primary: disable_audio_route:
reset and update mixer path: audio-ull-playback
03-11 01:14:01.182  2226  2748 V audio_hw_primary: disable_audio_route:
exit
03-11 01:14:01.182  2226  2748 D audio_hw_primary: disable_snd_device:
snd_device(2: speaker)
03-11 01:14:01.229  2226  2748 V audio_hw_primary: stop_output_stream:
exit: status(0)
// Disable usecase audio-ull playback (disable_audio_route) and disable
output sound device (disable_snd_device).
```

6.1.3 Kernel logs – Setup

```
01-01 04:22:13.864     0     0 D msm_pcm_routing_process_audio: reg 2 val 7
set 1
//Routing is set up between the MSM_FRONTEND_DAI_MULTIMEDIA8 (7) in the
msm-pcm-dsp-noirq platform driver and the backend SLIMBUS_0_RX (2) Codec
Device
01-01 04:22:13.867     0     0 D msm_pcm_routing_hw_params: BE Sample Rate
(48000) format (2) be_id 2
//hw_params for SLIMBUS_0_RX (BE_ID 2). Sample rate: 48khz and format 2
(SNDRV_PCM_FORMAT_S16_LE defined in include/uapi/sound/asound.h)
01-01 04:22:13.908     0     0 D msm_pcm_routing_prepare: substream->pcm-
>id:(Slimbus Playback)
//msm_pcm_routing_prepare "Slimbus Playback" stream ID is connected to
MSM_BACKEND_DAI_SLIMBUS_0_RX defined in kernel/msm-
4.4/sound/soc/msm/msm8998.c

01-01 04:22:13.910     0     0 D __afe_port_start: port id: 0x4000
```

Qualcomm Confidential - May Contain Trade Secrets 2023-01-05 10:34:23 GMT nijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
// TX AFE port 0x4000 is started defined in kernel/msm-
4.4/include/sound/apr_audio.h
//#define SLIMBUS_0_RX     0x4000        /* index = 15 */

01-01 04:22:13.926     0     0 D msm_pcm_trigger: P Trigger start
01-01 04:22:13.927     0     0 D event_handler: Payload =
[0x10daa]stat[0x0]
01-01 03:09:49.073     0     0 D ptr(P)  : appl = 672, hw = 480 read_index
= 1920
// msm_pcm_pointer queries the read_index from q6asm_get_shared_pos. hw ptr
converts the read_index from bytes to frames. And in this case the app
pointer is basically 192 frames (1ms) ahead of the hw ptr.
```

If all goes as expected, the application pointer always maintains a lead of 1 ms ahead of the apps pointer.

6.1.4 Kernel logs – Tear down

```
12-31 17:49:40.705     0     0 D msm_pcm_trigger: SNDRV_PCM_TRIGGER_STOP
//Playback is stopped by sending SNDRV_PCM_TRIGGER_STOP

12-31 17:49:40.832     0     0 D afe_close: port_id = 0x4000
12-31 17:49:40.840     0     0 D msm_pcm_routing_close: substream->pcm-
>id:(Slimbus Playback)
12-31 17:49:40.882     0     0 D msm_pcm_routing_process_audio: reg 2 val 7
set 0

//Disabling routing between the MSM_FRONTEND_DAI_MULTIMEDIA8 (7) in the
msm-pcm-dsp-noirq platform driver and the backend SLIMBUS_0_RX (2) Codec
Device
```

6.2 Log analysis – LL playback

The following log snippets indicate what to look for in the logcat and kernel logs.

6.2.1 Logcat logs – Setup

6.2.1.1 Audio track creation

```
03-07 01:27:20.455   4358  4358 V AudioTrack: set(): streamType 8,
sampleRate 48000, format 0x1, channelMask 0x1, frameCount 960, flags #4,
notificationFrames 0, sessionId 0, transferType 1, uid -1, pid -1
//StreamType:
(/system/media/audio/include/system/audio.h(audio_stream_type_t)
Format: 0x1 (/system/media/audio/include/system/audio.h (audio_format_t))
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
Channels: 0x1
(/system/media/audio/include/system/audio.h(audio_channel_mask_t))
Flags: 4 (/system/media/audio/include/system/audio.h(audio_output_flags_t))

03-07 01:27:20.456   989  1180 V AudioPolicyManagerCustom:
getNewOutputDevice() selected device 2
//Output device gets selected as speaker
(/system/media/audio/include/system/audio.h(audio_device_type_t))
```

6.2.1.2 Audio HAL

```
03-07 01:27:20.460   989  1414 D audio_hw_primary: start_output_stream:
enter: stream(0xe6dd3780)usecase(1: low-latency-playback) devices(0x2)
//Output Stream selects use case as “low-latency-playback” per the code
below in adev_open_output_stream() (defined in
/hardware/qcom/audio/hal/audio_hw.c)
else if (out->flags & AUDIO_OUTPUT_FLAG_FAST) {
            out->usecase = USECASE_AUDIO_PLAYBACK_LOW_LATENCY;
            out->config = pcm_config_low_latency;

out->usecase gets set to USECASE_AUDIO_PLAYBACK_LOW_LATENCY which
corresponds to “low-latency-playback”

03-07 01:27:20.461   989  1414 D audio_hw_primary: select_devices for use
case (low-latency-playback)
03-07 01:27:20.461   989  1414 D audio_hw_primary: select_devices:
out_snd_device(2: speaker) in_snd_device(0: )
03-07 01:27:20.462   989  1414 D audio_hw_primary: enable_snd_device:
snd_device(2: speaker)
03-07 01:27:20.501   989  1414 V audio_hw_primary: enable_audio_route: exit
03-07 01:27:20.501   989  1414 D audio_hw_primary: select_devices: done
// For the use-case low-latency playback. select_devices the output device
gets selected
snd_device = SND_DEVICE_OUT_SPEAKER
select_devices enables the device (enable_snd_device)
select_devices also enables the audio route for low-latency playback.

03-07 01:27:20.501   989  1414 V audio_hw_primary: start_output_stream:
Opening PCM device card_id(0) device_id(13) format(0)
//Opening PCM Device 0
03-07 01:27:20.586   989  1414 D audio_hw_primary: start_output_stream:
exit
//Output stream started successfully.
03-07 01:27:20.586   989  1414 V audio_hw_primary: out_write: writing
buffer (0 bytes) to pcm device
03-07 01:27:20.587   989  1413 V audio_hw_primary: out_write: writing
buffer (768 bytes) to pcm device
```

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:33 GMT nijasher@qualcomm.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
03-07 01:27:20.587   989  1413 V audio_hw_primary: out_write: writing
buffer (768 bytes) to pcm device
03-07 01:27:20.587   989  1413 V audio_hw_primary: out_write: writing
buffer (768 bytes) to pcm device
// Buffers are transferred every 4 ms
```

After every write, the apps processor waits for an event from the DSP to indicate that it has consumed a buffer and freed up a buffer. This triggers the apps processor to write the next buffer into the memory (see Section 3.2).

6.2.2 Logcat logs – Tear down

```
03-07 15:34:54.276   985  1464 D audio_hw_primary: out_standby: enter:
stream (0xedad3a00) usecase(1: low-latency-playback)
03-07 15:34:54.459   985  1464 D audio_hw_primary: disable_audio_route:
reset and update mixer path: low-latency-playback
03-07 15:34:54.465   985  1464 D audio_hw_primary: disable_snd_device:
snd_device(2: speaker)
// Disable usecase low-latency-playback (disable_audio_route) and disable
output sound device (disable_snd_device).
```

6.2.3 Kernel logs – Setup

```
01-03 00:33:20.657     0     0 D msm_pcm_routing_process_audio: reg 2 val 4
set 1
//Routing is set up between the MSM_FRONTEND_DAI_MULTIMEDIA5 (4) in the
msm-pcm-dsp-noirq platform driver and the backend SLIMBUS_0_RX (2) Codec
Device
01-03 00:33:20.659     0     0 D msm_pcm_routing_hw_params: BE Sample Rate
(48000) format (2) be_id 2
//hw_params for SLIMBUS_0_RX (BE_ID 2). Sample rate: 48khz and format 2
(SNDRV_PCM_FORMAT_S16_LE defined in include/uapi/sound/asound.h)
01-03 00:33:20.686     0     0 D msm_pcm_routing_prepare: substream->pcm-
>id:(Slimbus Playback)
//msm_pcm_routing_prepare "Slimbus Playback" stream ID is connected to
MSM_BACKEND_DAI_SLIMBUS_0_RX defined in kernel/msm-
4.4/sound/soc/msm/msm8998.c
01-03 00:11:17.781     0     0 D __afe_port_start: port id: 0x4000
// TX AFE port 0x4000 is started defined in kernel/msm-
4.4/include/sound/apr_audio.h
//#define SLIMBUS_0_RX     0x4000        /* index = 15 */

01-03 00:33:20.697     0     0 D         : msm_pcm_playback_prepare
01-03 00:33:20.705     0     0 D msm_pcm_routing_reg_phy_stream: setting
idx bit of fe:4, type: 0, be:2
01-03 00:33:20.715     0     0 D msm_pcm_playback_copy: prtd->out_count = 2
01-03 00:33:20.715     0     0 D msm_pcm_playback_copy: fbytes =768:
xfer=768 size=768
```

Qualcomm Confidential - May Contain Trade Secrets 2022-11-16 10:34:33 GMT nijasheng@hongcheer.com

```
01-03 00:33:20.715     0     0 D msm_pcm_playback_copy: fbytes = 0:
xfer=768

01-03 00:33:20.715     0     0 D msm_pcm_trigger: Trigger start

01-03 00:33:20.716     0     0 D         : pcm_irq_pos = 0
01-03 00:33:20.716     0     0 D msm_pcm_playback_copy: prtd->out_count = 1
01-03 00:33:20.716     0     0 D msm_pcm_playback_copy: fbytes =768:
xfer=768 size=768
01-03 00:33:20.716     0     0 D msm_pcm_playback_copy: fbytes = 0:
xfer=768
01-03 00:33:20.716     0     0 D         : pcm_irq_pos = 0
```

```
// Low latency playback is started by calling and setting the hardware
params and calling msm_pcm_playback_prepare followed by the
msm_pcm_trigger() function.
```

PCM data is copied to DSP buffers. There are two buffers, each 768 bytes.

```
01-03 00:33:20.717     0     0 D event_handler: writing 768 bytes of buffer
to dsp
01-03 00:33:20.717     0     0 D event_handler: writing 768 bytes of buffer
to dsp
01-03 00:33:20.721     0     0 D         : ASM_DATA_EVENT_WRITE_DONE_V2
01-03 00:33:20.721     0     0 D         : Buffer Consumed = 0x1009a000
// Once the buffer is consumed, pcm_irq_pos increments by period size that
is 768

01-03 00:33:20.721     0     0 D         : pcm_irq_pos = 768
01-03 00:33:20.721     0     0 D msm_pcm_playback_copy: prtd->out_count = 1
01-03 00:33:20.721     0     0 D msm_pcm_playback_copy: fbytes =768:
xfer=768 size=768
01-03 00:33:20.721     0     0 D msm_pcm_playback_copy: fbytes = 0:
xfer=768
01-03 00:33:20.721     0     0 D msm_pcm_playback_copy: writing 768 bytes
of buffer to dsp
01-03 00:33:20.721     0     0 D         : pcm_irq_pos = 768

01-03 00:33:20.725     0     0 D         : ASM_DATA_EVENT_WRITE_DONE_V2
01-03 00:33:20.725     0     0 D         : Buffer Consumed = 0x1009a300
01-03 00:33:20.725     0     0 D         : pcm_irq_pos = 0
```

```
// Once the entire PCM data in the two buffers is consumed, the pcm_irq_pos
is reset to 0 and incremented in steps of 768
```

```
01-03 00:33:20.725     0     0 D msm_pcm_playback_copy: prtd->out_count = 1
01-03 00:33:20.725     0     0 D msm_pcm_playback_copy: fbytes =768:
xfer=768 size=768
```

Qualcomm Confidential - May Contain Trade Secrets 2022-11-15 10:34:33 GMT rijlasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
01-03 00:33:20.725     0     0 D msm_pcm_playback_copy: fbytes = 0:
xfer=768
01-03 00:33:20.725     0     0 D msm_pcm_playback_copy: writing 768 bytes
of buffer to dsp
01-03 00:33:20.725     0     0 D         : pcm_irq_pos = 0

01-03 00:33:20.729     0     0 D         : ASM_DATA_EVENT_WRITE_DONE_V2
01-03 00:33:20.729     0     0 D         : Buffer Consumed = 0x1009a000
01-03 00:33:20.729     0     0 D         : pcm_irq_pos = 768
```

6.2.4 Kernel logs – Tear down

```
01-03 00:33:40.514     0     0 D         : SNDRV_PCM_TRIGGER_STOP
01-03 00:33:40.518     0     0 D msm_pcm_playback_close: CMD_EOS timeout is
50
//Playback is stopped by sending the SNDRV_PCM_TRIGGER_STOP.
01-03 00:33:40.526     0     0 D         : ASM_DATA_EVENT_RENDERED_EOS
// Once all the buffers are consumed the DSP sends the EOS event.
01-03 00:33:40.538     0     0 D msm_pcm_routing_dereg_phy_stream:
copp:1,idx bit fe:4,type:0,be:2
01-03 00:33:40.563     0     0 D afe_close: port_id = 0x4000
// AFE Port ID is closed.
01-03 00:33:40.657     0     0 D msm_pcm_routing_process_audio: reg 2 val 4
set 0
//Routing disabled between the MSM_FRONTEND_DAI_MULTIMEDIA5 (4) in the msm-
pcm-dsp-noirq platform driver and the backend SLIMBUS_0_RX (2) Codec Device
```

Qualcomm Confidential - May Contain Trade Secrets 2022-11-15 10:34:33 GMT nijiashengpengcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

A References

A.1 Related documents

Title	Number
Qualcomm Technologies, Inc.	
Linux SMD GLink Migration Guide	80-VJ608-2
Non-HLOS SMDL To GLink Migration Guide	80-VJ608-3
GLink (Generic Link) Overview	80-P2598-1

A.2 Acronyms and terms

Acronym or term	Definition
ADM	Audio Device Manager
ASM	Audio Stream Manager
FE	Front End
LL	Low Latency
ULL	Ultra-Low Latency

Qualcomm Confidential May Contain Trade Secrets 2022-01-05 10:34:33 GMT nijiasheng@longcheer.com